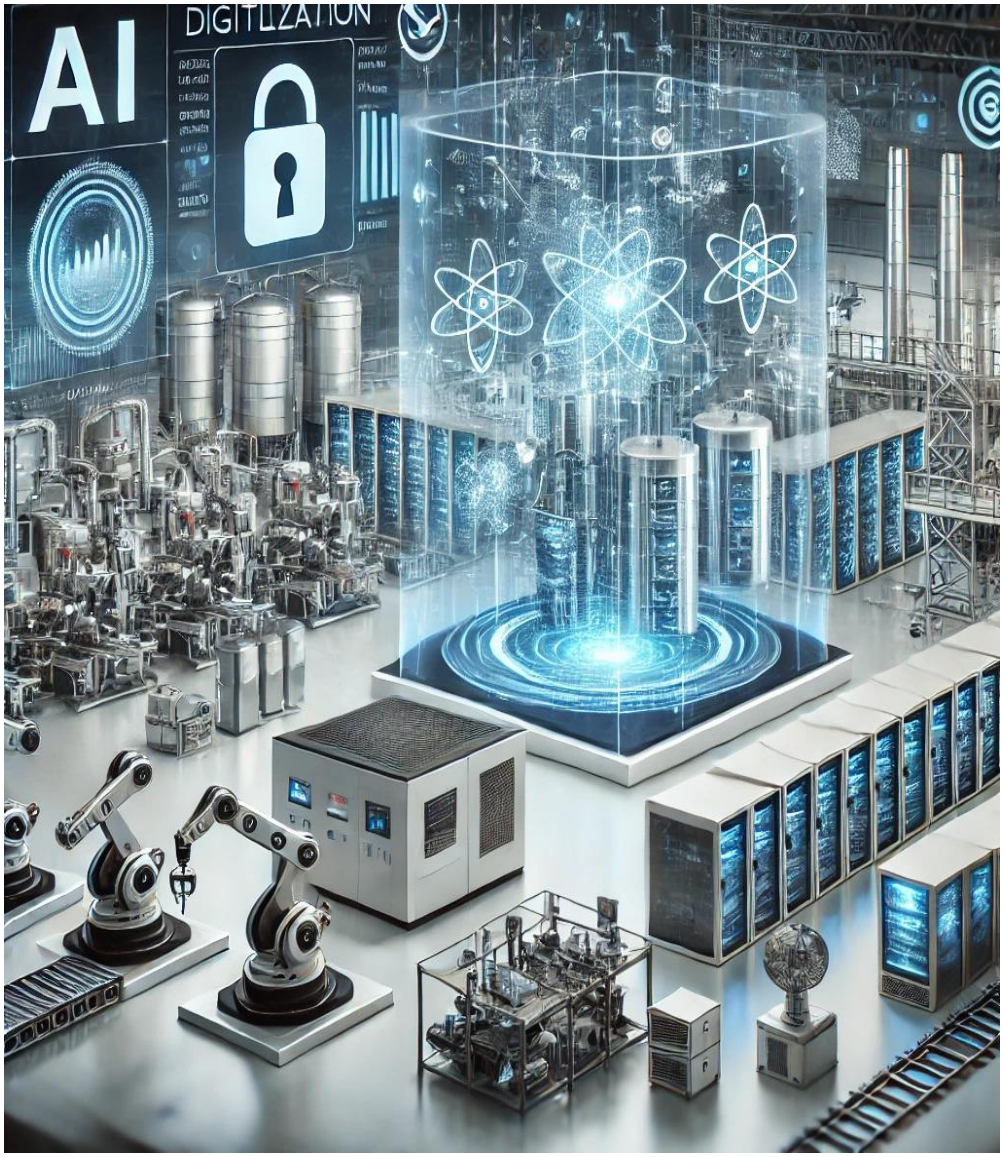
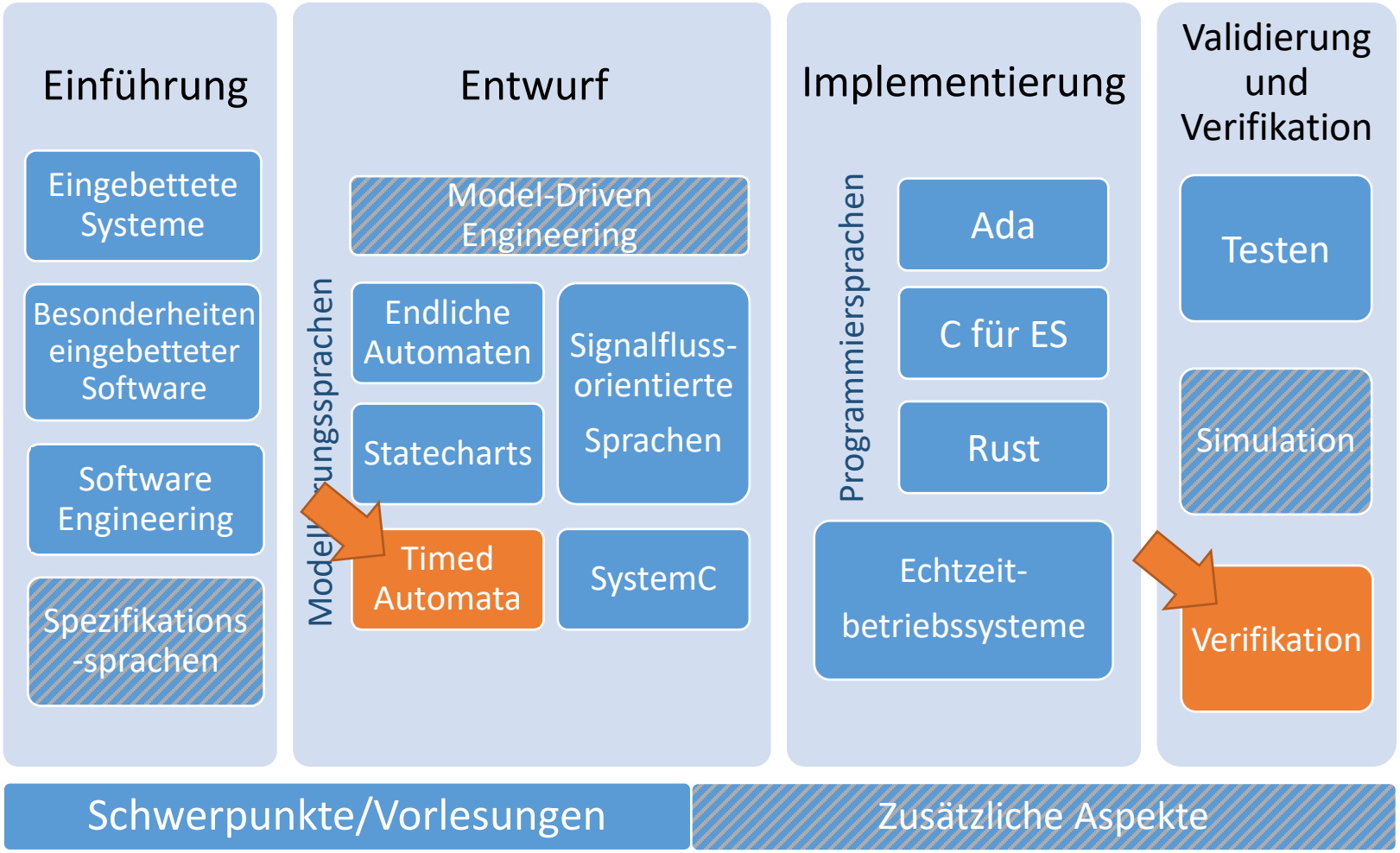



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

# Embedded Systems Engineering Verifikation

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

# Diese Vorlesung



# Inhalt

- Einführung
- Temporallogik
  - Grundlagen
  - UPPAAL Queries
- Verifikation von TA
- Fazit + Literatur

# Motivation

Bis jetzt: Validierung + Verifikation mit Tests

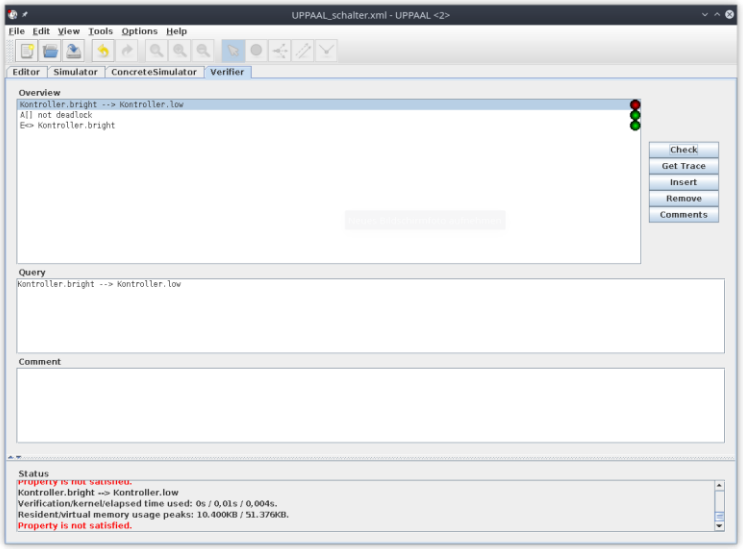
Tests haben **Grenzen:**

- **Explosion der Anzahl der Testfälle** bei Nebenläufigkeit
- **Unendliches Verhalten** nichtterminierender Systeme **nicht abgedeckt**
- Tests können nur Fehler finden, **nie ausschließen!**
- Bestimmte Eigenschaften sind **nicht bzw. schwer testbar.** Beispiele:
  - Keine Deadlocks vorhanden
  - Es ist *immer* möglich, das System in einen sicheren Operationsmodus zu bringen

# Modelchecking

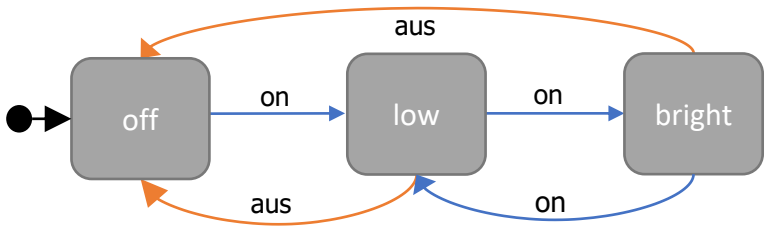
**Modelchecker** prüfen einen endlichen Zustandsraum vollständig auf mögliche Verletzung einer Spezifikation:

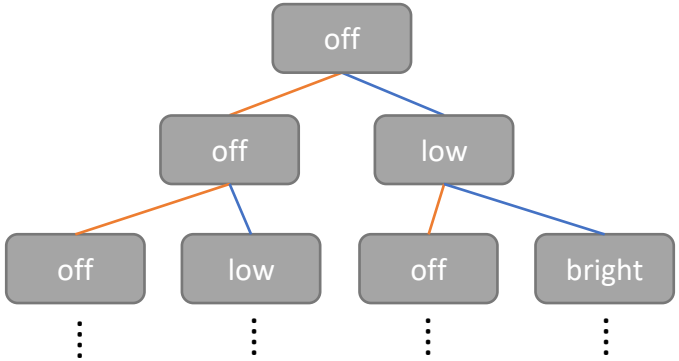
- mathematisch fundiert
- vollautomatisch
- In UPPAAL für Timed Automata implementiert



# Modelchecking mit Timed Automata

Das **endliche Transitionssystem** eines endlichen Automaten *(ohne Zeit)* generiert einen **unendlichen Baum** (mögliche Pfade):




Das endliche Modell deckt also unendlich viele, beliebig lange Pfade ab.

Bei einem endlichen Transitionssystem sucht Model Checking im **gesamten Zustandsraum** nach einen **Gegenbeispiel für eine Aussage**.

Wenn kein Gegenbeispiel gefunden wird, ist die Aussage wahr.

Da alle Möglichkeiten überprüft wurden.

→ Bei großen Modellen sehr **rechenintensiv**

Für Timed Automata muss Zeit diskretisiert werden

- Mehr dazu später

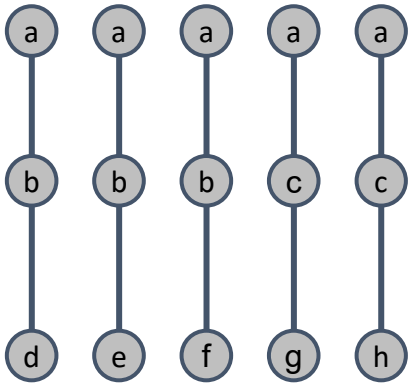
# Inhalt

- Einführung
- Temporallogik
  - Grundlagen
  - UPPAAL Queries
- Verifikation von TA
- Fazit + Literatur

# Eigenschaften

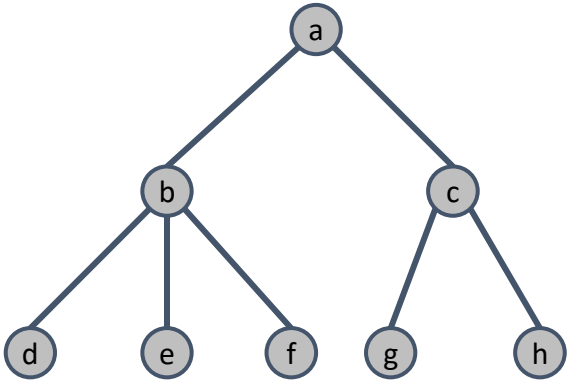
Eigenschaften von TA werden als zeitliche Eigenschaften über Pfaden formuliert:

**Linear-time Temporal Logic (LTL)**




**Lineare** (total geordnete) Folge von Zeitpunkten. LTL spricht über Menge einzelner Pfade.

**Computation Tree Logic (CTL)**




**Verzweigte** Zeit: verschiedene Folgezustände möglich. CTL spricht über den Berechnungsbaum (computation tree).

Timed Automata: zusätzlich Zeiteinschränkungen ⇨ Timed CTL (TCTL)

# Linear-time Temporal Logic

- Beschreibt Aussagen über einzelne Pfade oder eine Menge von Pfaden
- Eine LTL-Formel ist genau dann auf einem Pfad erfüllt, wenn sie auf Position 0 des Pfades erfüllt ist.
- Operatoren der Aussagenlogik werden durch ne**X**t, **F**inally, **G**lobally und **U**ntil ergänzt.

# LTL Syntax

Induktive Definition von LTL Formeln:

$$\phi ::= \quad \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi_1 \wedge \phi_2) \mid (\phi_1 \vee \phi_2) \mid (\phi_1 \Rightarrow \phi_2) \mid$$
$$X\,\phi \mid F\,\phi \mid G\,\phi \mid \phi_1\, U\, \phi_2 \mid \phi_1\, W\, \phi_2$$

- $\bot$ und $\top$ stehen für falsch und wahr
- p steht für eine atomare Formel (Prädikat)
- die aussagenlogischen Verknüpfungen sind die üblichen
- **Temporale Operatoren:**

kann auch der aktuelle Zustand sein

**X** $\phi$ "$\phi$ muss im nächsten (ne**X**t) Zustand gelten"

**F** $\phi$ "$\phi$ muss irgendwann in der Zukunft (**F**uture/**F**inally) gelten"

**G** $\phi$ "$\phi$ muss **G**lobal in all künftigen Zuständen gelten"

$\phi_1$ **U** $\phi_2$ "$\phi_1$ gilt in allen Zuständen bis (**U**ntil) $\phi_2$ irgendwann gilt"

$\phi_1$ **W** $\phi_2$ "$\phi_1$ gilt in allen Zuständen bis $\phi_2$ irgendwann gilt oder $\phi_2$ gilt niemals" (**W**eak until)

Zum Klammersparen: ¬, X, G, F binden starker als U und dann ∧ und ∨.

# LTL Formeln: Übung

- Welche der LTL Formeln X *p*, F *p* und G *p* gelten auf folgenden Pfaden?

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | F | T | F |

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | F | F | T |

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | T | F |

| | 0 | 1 | 2 | |
|---|---|---|---|---|
| *p* | T | T | T | … |

- Welche der LTL Formeln *p* U *q* und *p* W *q* gelten auf folgenden Pfaden?

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | T | F |
| *q* | F | F | T |

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | F | T |
| *q* | F | F | T |

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | F | T |
| *q* | F | F | F |

# LTL Formeln: Übung

- Welche der LTL Formeln X *p*, F *p* und G *p* gelten auf folgenden Pfaden?

gilt: X p, F p,
gilt nicht: G p

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | F | T | F |

gilt: F p,
gilt nicht: X p, G p

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | F | F | T |

gilt: X p, F p,
gilt nicht: G p

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | T | F |

gilt: X p, F p, G p

| | 0 | 1 | 2 | |
|---|---|---|---|---|
| *p* | T | T | T | … |

- Welche der LTL Formeln *p* U *q* und *p* W *q* gelten auf folgenden Pfaden?

gilt: p U q, p W q

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | T | F |
| *q* | F | F | T |

gilt nicht: p U q, p W q

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | F | T |
| *q* | F | F | T |

gilt: p W q
gilt nicht: p U q

| | 0 | 1 | 2 |
|---|---|---|---|
| *p* | T | F | T |
| *q* | F | F | F |

# CTL Syntax

Induktive Definition:

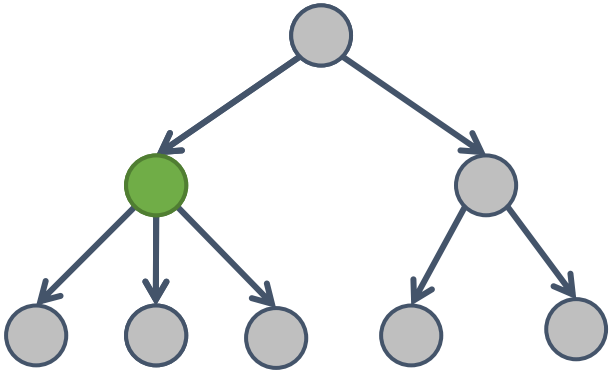
$$\phi ::= \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \Rightarrow \phi) \mid$$
$$AX\,\phi \mid EX\,\phi \mid AG\,\phi \mid EG\,\phi \mid$$
$$AF\,\phi \mid EF\,\phi \mid A\,[\phi_1\,U\,\phi_2] \mid E\,[\phi_1\,U\,\phi_2]$$

Jeder Temporal-operator wird durch genau einen Pfadquantor quantifiziert

- $\bot$ und $\top$ stehen für falsch und wahr (**T**rue)
- p steht für eine atomare Formel (**P**rädikat)
- die aussagenlogischen Verknüpfungen sind die üblichen
- temporallogische Verknüpfungen:
  - A "entlang aller Pfade" (**A**lways)
  - E "entlang mindestens eines Pfades" (**E**xists)
  - X "nächster Zustand" (ne**X**t)
  - F "im Anfangszustand oder in **einem** zukünftigen Zustand" (**F**uture)
  - G "im Anfangszustand und in **allen** zukünftigen Zuständen" (**G**lobally)
  - U "bis" (**U**ntil)

# Temporale Operatoren in CTL

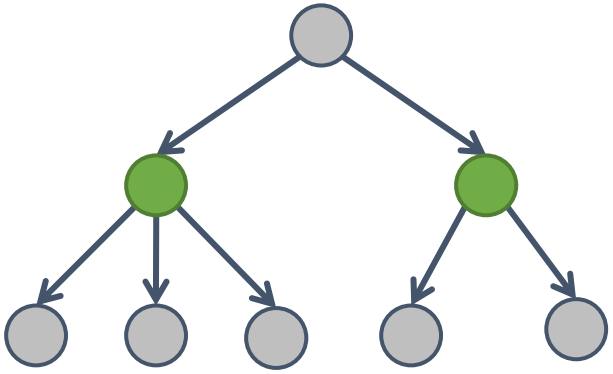
- Die Syntax der Aussagenlogik wird mit temporalen Operatoren erweitert.
  - Im nächsten (neXt) Zustand EX $P$, AX $P$
  - Irgendwann (Future) EF $P$, AF $P$
  - Immer (Global) EG $P$, AG $P$
  - Von nun an bis (Until) E$[P$ U $Q]$, A$[P$ U $Q]$



EX $P$

mindestens

Es **existiert** ein Pfad auf dem $P$ im **nächsten** Zustand gilt

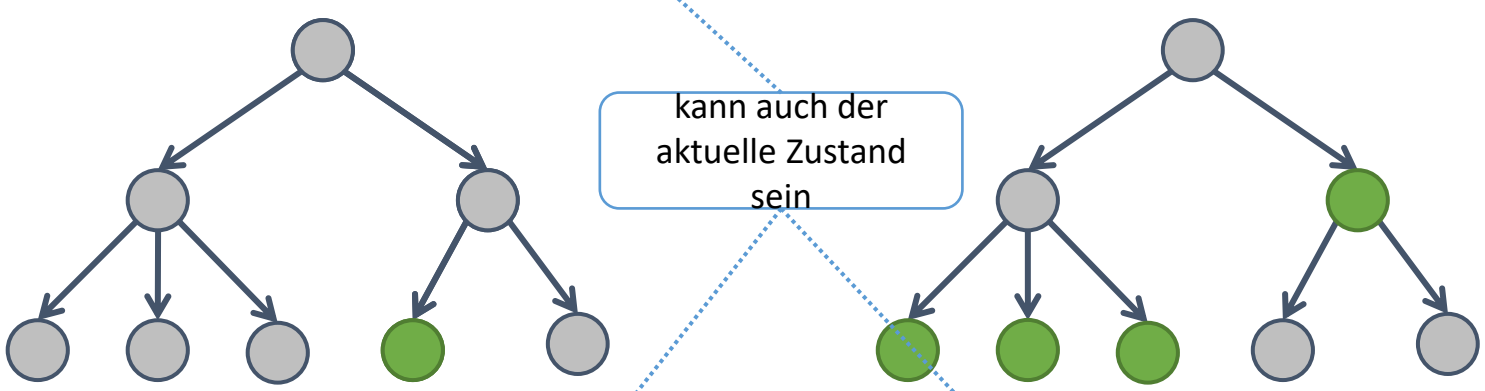


AX $P$

Entlang **aller** Pfade gilt $P$ im **nächsten** Zustand

# Temporale Operatoren in CTL

- Die Syntax der Aussagenlogik wird mit temporalen Operatoren erweitert.
  - Im nächsten (neXt) Zustand EX $P$, AX $P$
  - Irgendwann (Future) EF $P$, AF $P$
  - Immer (Global) EG $P$, AG $P$
  - Von nun an bis (Until) E$[P$ U $Q]$, A$[P$ U $Q]$




EF $P$

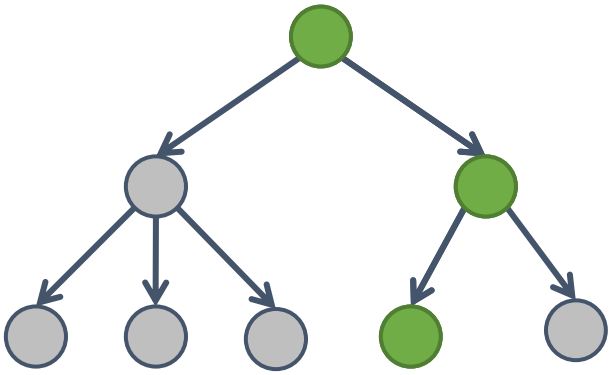
Es **existiert** ein Pfad auf dem $P$ in irgendeinem **zukünftigen** Zustand gilt

AF $P$

Entlang **aller** Pfade gilt $P$ in irgendeinem **zukünftigen** Zustand

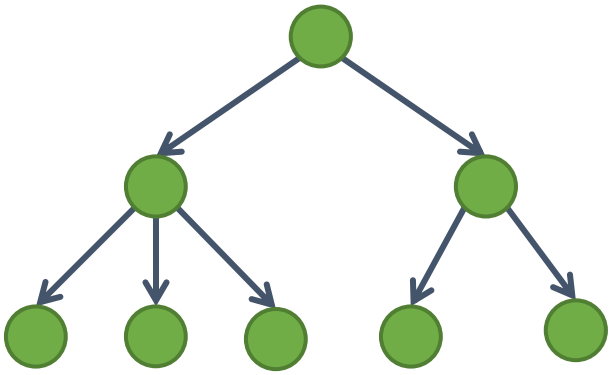
# Temporale Operatoren in CTL

- Die Syntax der Aussagenlogik wird mit temporalen Operatoren erweitert.
  - Im nächsten (neXt) Zustand EX $P$, AX $P$
  - Irgendwann (Future) EF $P$, AF $P$
  - Immer (Global) EG $P$, AG $P$
  - Von nun an bis (Until) E$[P$ U $Q]$, A$[P$ U $Q]$



EG $P$

Es **existiert** ein Pfad, entlang dem $P$ **global** gilt (=in allen Zuständen auf dem Pfad)

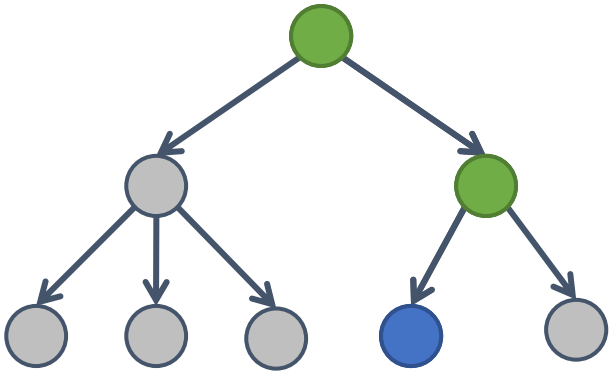


AG $P$

Entlang **aller** Pfade gilt $P$ **global** (= gilt immer)

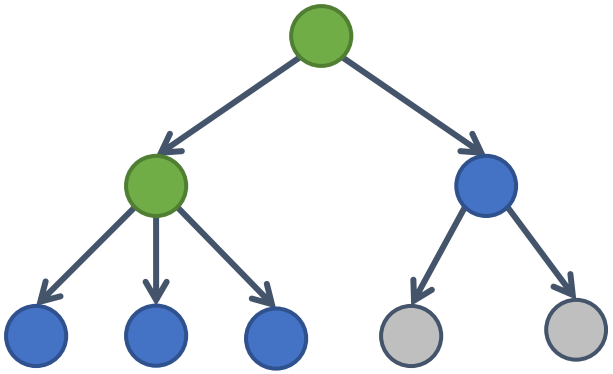
# Temporale Operatoren in CTL

- Die Syntax der Aussagenlogik wird mit temporalen Operatoren erweitert.
  - Im nächsten (neXt) Zustand EX $P$, AX $P$
  - Irgendwann (Future) EF $P$, AF $P$
  - Immer (Global) EG $P$, AG $P$
  - Von nun an bis (Until) E$[P$ U $Q]$, A$[P$ U $Q]$



$\mathrm{E}[P \text{ U } Q]$

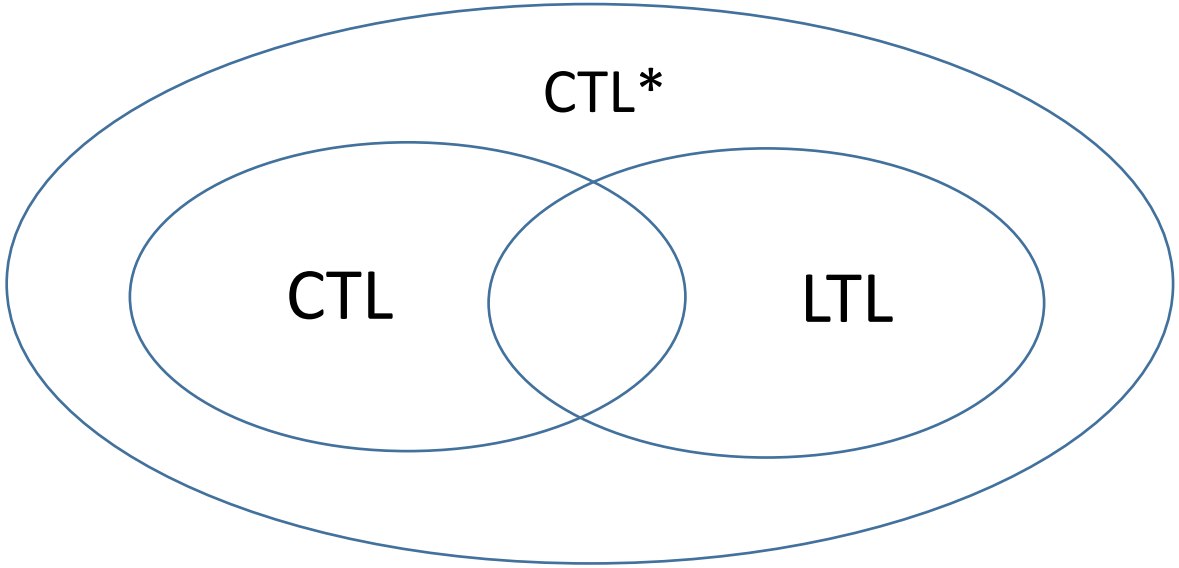
Auf min. einem Pfad gilt ab jetzt $P$, **bis** irgendwann $Q$ gilt



$\mathrm{A}[P \text{ U } Q]$

Entlang aller Pfade gilt ab jetzt $P$, bis irgendwann $Q$ gilt

# Abgrenzung CTL - LTL




- In CTL und nicht in LTL: AG EF p
- In LTL und nicht in CTL: GF p $\Rightarrow$ F q
- Nicht in CTL und LTL: E (GF p)

# Timed CTL

- TCTL: Erweiterung von CTL um Zeitbeschränkungen

Uhr wird für Formel gestartet und durch $\simeq d$ beschränkt

- Syntax: $\phi ::= \bot \mid \top \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \Rightarrow \phi) \mid$

$AG_{\simeq d}\ \phi \mid EG_{\simeq d}\ \phi \mid AF_{\simeq d}\ \phi \mid EF_{\simeq d}\ \phi \mid$

$A\ [\phi_1\ U_{\simeq d}\ \phi_2] \mid E\ [\phi_1\ U_{\simeq d}\ \phi_2]$

$d$ **ganzzahlig,**
$\simeq\ \in \{<, \leq, =, \geq, >\}$,
auch Intervalle möglich:
z.B. [d1,d2]

$\simeq d$ : entspricht Uhrenbedingungen in Timed Automata, p kann auch eine Uhrenbedingung sein (z.B. $x > 10$)

- mit TCTL können Eigenschaften der Form: "etwas geschieht frühestens nach 10 ms und spätestens nach 200 ms" ausgedrückt werden

  Beispiel: wenn Alarm ausgelöst wird, geht das System innerhalb von 160 ms in einen Sicherheitszustand über

$$alarm \Rightarrow AG_{(\leq 160)}\ safety_mode$$

➢ TCTL-Formeln über Timed Automata dürfen Zustände (locations), logische Operatoren und Uhrenbedingungen enthalten

# Inhalt


- Einführung
- Temporallogik
  - Grundlagen
  - UPPAAL Queries
- Verifikation von TA
- Fazit + Literatur

# UPPAAL Queries (1)

**Pfadquantoren**:

- E<> p — auf mindestens einem Ausführungspfad gilt p irgendwann (entspricht EF p)
- A<> p — auf allen Ausführungspfaden gilt p irgendwann (entspricht AF p)
- E[] p — auf mindestens einem Ausführungspfad gilt p immer (entspricht EG p)
- A[] p — auf allen Ausführungspfaden gilt p immer (entspricht AG p)
- p --> q — wenn p auf einem Ausführungspfad gilt, gilt irgendwann später auch q (*Leads-To*-Operator, entspricht AG (p ⇒ AF q) )

Pfadquantoren dürfen in UPPAAL nicht geschachtelt werden!

# UPPAAL Queries (2)

A[] expr | E[] expr | A<> expr | E<> expr | expr --> expr

Syntax von Expressions ähnlich C/C++

- Variablenzugriffe
- and, or, not, imply, ==
- forall and exists (für alle Arrayelemente gilt)
- deadlock (A[] not deadlock)
- Uhrenbedingungen
- Zustände (Automat X in Zustand y = X.y)

Expressions dürfen keine Seiteneffekte haben!

# Syntax-Übersicht: UPPAAL vs. CTL

| CTL | UPPAAL |
|---|---|
| F | < > |
| G | [ ] |
| AG (p ⇒ AF q) | p --> q |
| ¬ p | not p (oder !p) |
| p ∧ q | p and q (oder p && q) |
| p ∨ q | p or q (oder p \|\| q) |
| p ⇒ q | p imply q |

# Typische UPPAAL Queries

Einige Eigenschaften müssen häufig verifiziert werden:

**Erreichbarkeit**

`E<> D.s` bzw. `A<> D.s`

Zustand s des Automaten D wird auf mindestens einem Pfad des Transitionssystems bzw. immer erreicht.

**Lebendigkeit** *(„Etwas Gutes wird immer passieren“)*

`D.s --> z==3` *In CTL: AG(D.s => AF D.z==3)*

Wenn s in D erreicht wird, wird irgendwann z den Wert 3 annehmen.

**Sicherheit** *(„Etwas Schlechtes wird nie passieren/etwas Gutes ist immer wahr.“)*

`E[] b < 10` bzw. `A[] b < 10`

Der Wert von b kann bzw. muss dauerhaft kleiner als 10 bleiben.

# Inhalt

- Einführung
- Temporallogik
  - Grundlagen
  - UPPAAL Queries
- Verifikation von TA
- Fazit + Literatur

# Zeit: diskret vs. kontinuierlich

## Diskrete Zeit

- Zeit wird in diskreten Schritten gezählt
- kann auf einen Takt (clock ticks) abgebildet werden
- kann auf endliche Variablen abgebildet werden (wenn Ausführungszeit endlich)

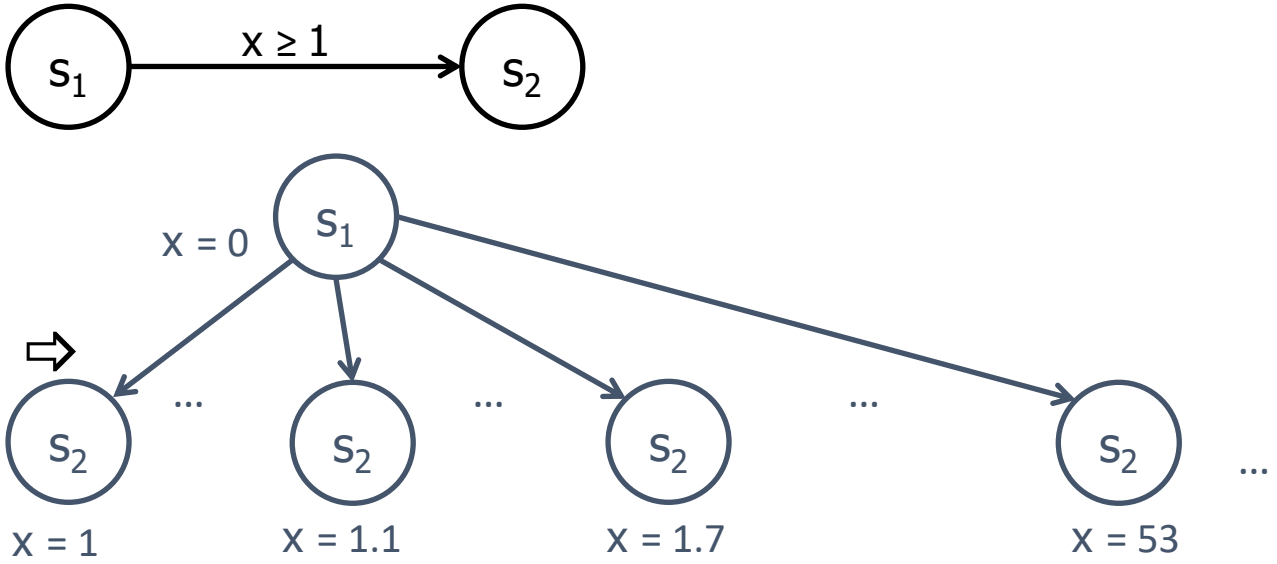
## Kontinuierliche Zeit

- kontinuierlicher Zeitverlauf
- Ereignisse können zu beliebigen Zeitpunkten eintreten
- Zeit kann um beliebige Werte erhöht werden
- Wertebereich: positive reelle Zahlen ⇨ unendlich

*Wie können wir prüfen, ob eine bestimmte Eigenschaft innerhalb eines Timed Automaton gilt?*

# Analyse von Timed Automata

Das Transitionssystem eines Timed Automaton hat einen unendlichen Zustandsraum

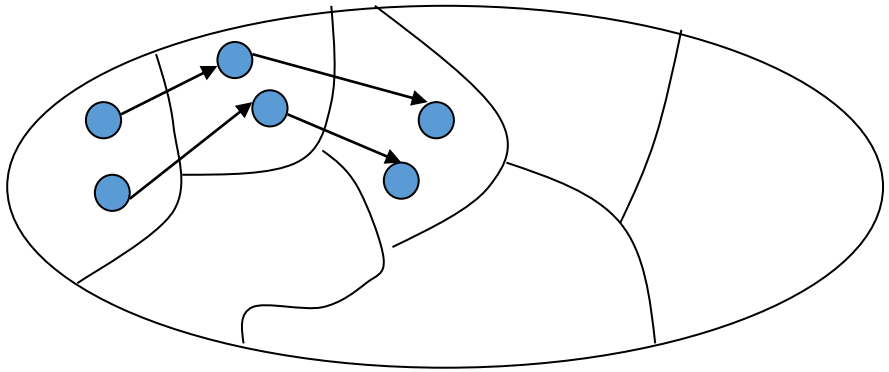



*Wie prüft man z.B. Eigenschaften der Form „auf allen Pfaden gilt..“?*

# Äquivalenzklassen

**Idee:**

- Häufig ist man nicht am exakten Wert der Uhrenbelegung, sondern nur an einem Zeit-Intervall interessiert (z.B.: die Übertragungszeit beträgt min. 20 ms und max. 50 ms)
- Dann kann der Wertebereich der Uhren auf eine (endliche) Menge von Zeit-Intervallen reduziert werden
- **Daher**: Partitionierung des unendlichen Zustandsraums in endlich viele Partitionen (**Regionen**)



# Quotientenautomat

**Partitionierung** der Zustandsmenge:

- Abstraktion von bestimmten Variablen (Wertebereich reduzieren oder weglassen)
- Zustände im Quotientenautomat sind Äquivalenzklassen von Zuständen im ursprünglichen Automaten
- Problem: Menge möglicher Abläufe kann sich vergrößern
- Gesucht: Abstraktion, die die Sprache nicht vergrößert

**Bei Timed Automata**:

- Nicht jede mögliche Uhrenbelegung führt zu einer Änderung des Ablaufs
- Manche Zustände sind “zeitlich abstrakt” äquivalent

# Äquivalenzklassenbildung

**Gesucht**: Äquivalenzklassen mit “ähnlichen” zeitlichen Eigenschaften

**Idee**: wenn es nur ganzzahlige Konstanten im Automaten gibt, spielen fraktionale Anteile der Uhren für die Auswertung der Uhrenbedingungen keine Rolle!

**Beispiel**: für die Auswertung von $x \geq 1$ spielt es keine Rolle, ob
$x = 1$, $x = 1.1$, $x = 1.7$, $x = \pi$, $x = 53$, …

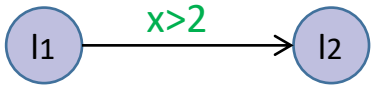
# Regionenäquivalenz: Intuition

Ansatz zur Partitionierung des Zustandsraums: **Regionen**

**Idee** von äquivalenten Uhrenbelegungen:

- Wert des fraktionalen Anteils einer Uhr nicht entscheidend
- Größenordnung der fraktionalen Anteile zweier Uhren muss gleich bleiben
- max. Wert für eine Uhr ($d_{max}$) aus vorhandenen Uhrenbedingungen gibt obere Grenze, größere Werte uninteressant

**Beispiel**:




- $(l_1, x < 1)$, $(l_1, x = 1)$, $(l_1, 1 < x < 2)$, $(l_1, x = 2)$, $(l_1, x > 2)$
- $(l_2, x > 2)$

# Beispiel: Äquivalenz von Uhrenbelegungen

**Annahme:**

- Uhrenvariablen: $x, y, z$
- $x_{max} = y_{max} = z_{max} = 3$
- Uhrenbelegungen $v: \{x, y, z\} \to \mathbb{R}^+$
  durch Vektoren $[x = d_1, y = d_2, z = d_3]$ repräsentiert

**Beispiel**

Werte von y irrelevant, da $y > y_{max}$

- $[1, 2.1, 2.423] \cong [1, 2.422, 2.423] \cong [1, 2.9, 2.95]$
- $[1, 3.4, 2.423] \cong [1, 3.5, 2.423] \cong [1, 42.12345, 2.95]$

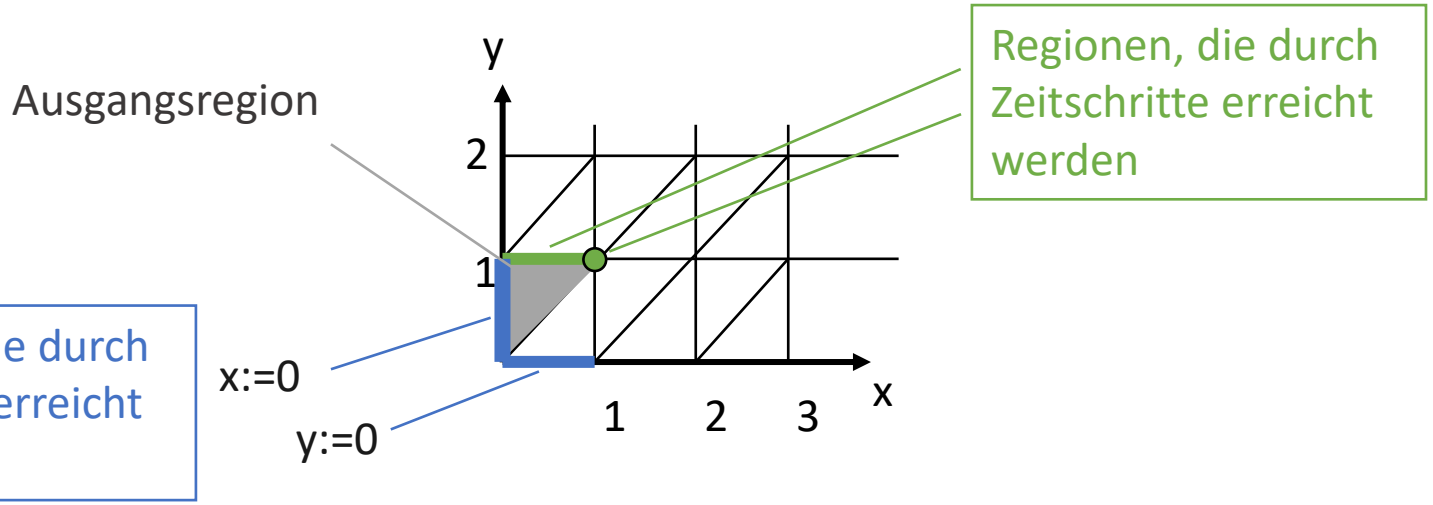
x=1 vs 1<x<2

- $[1, 2.1, 2.423] \ncong [1.1, 2.1, 2.423]$
- $[1, 2.1, 1.423] \ncong [1, 2.12, 1.1]$

|y|< |z| vs |y|>|z|

# Regionengraph

- Zustände des Regionengraphen: Tupel aus *locations* und *regions*
- Sukzessiver Aufbau: jeweils finden aller möglichen Folgezustände
  1. Zustände, die durch diskrete Schritte (Transitionen) erreicht werden: es vergeht keine Zeit, Uhren können zurückgesetzt werden
  2. Zustände, die durch Zeitschritte erreicht werden: Bewegung nach rechts oben (x und y nehmen gleichermaßen zu)

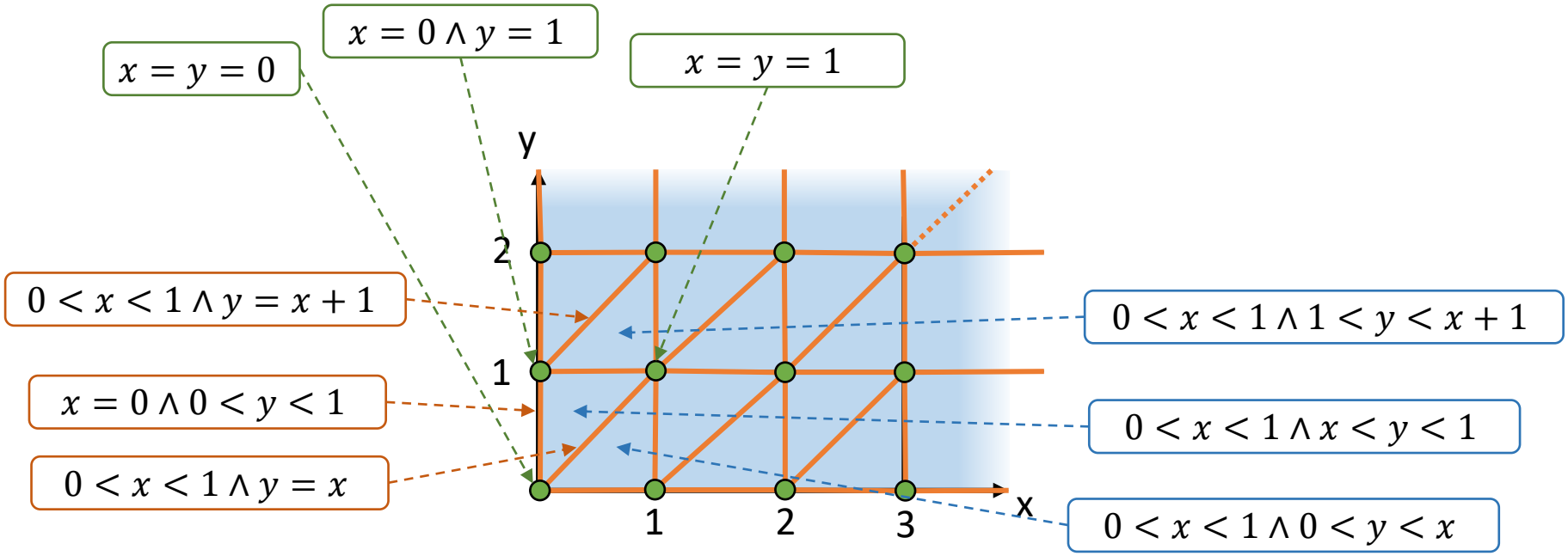


# Regionen mit mehreren Uhren

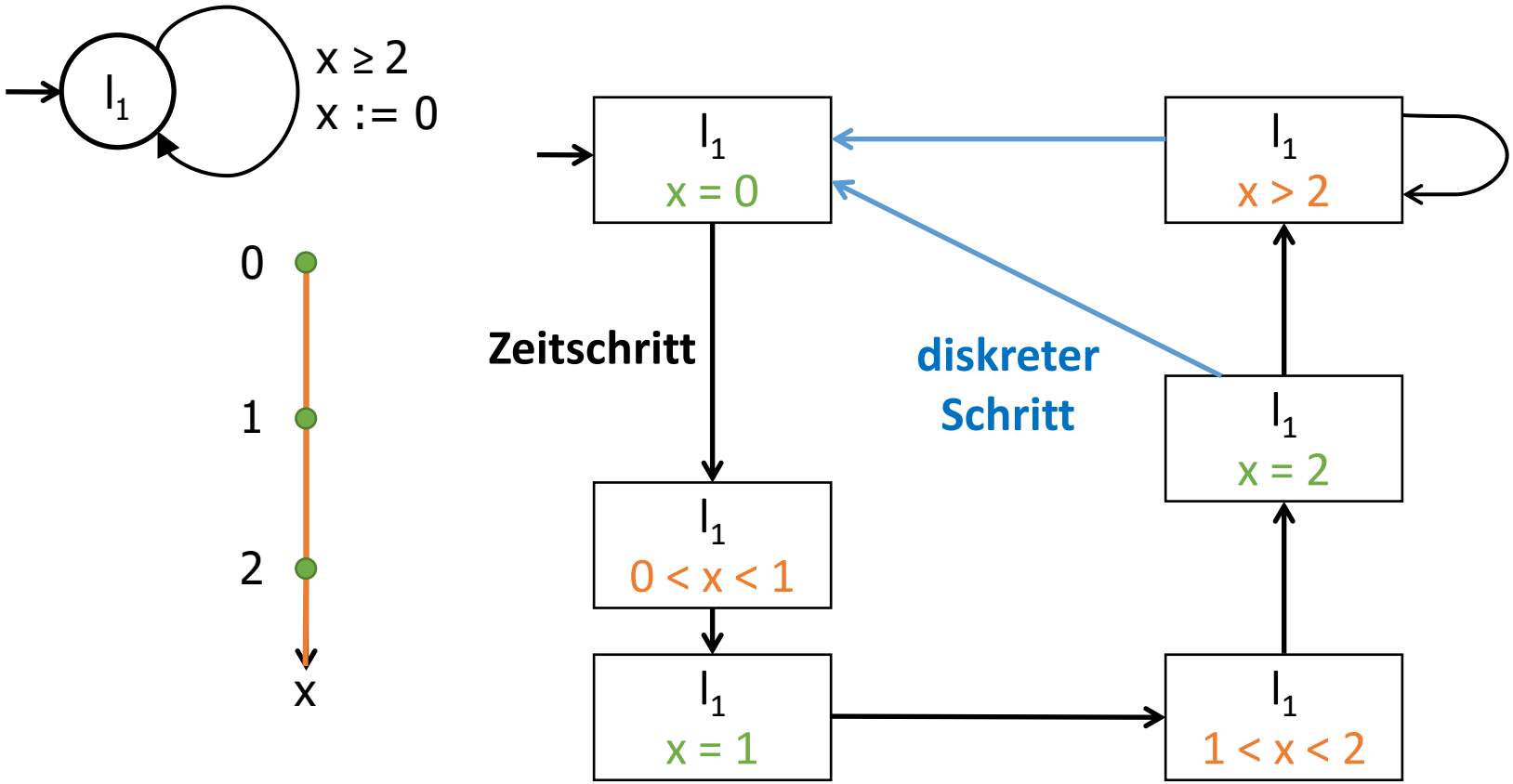
Bei mehreren Uhren muss man nicht nur die Wertebereiche der Uhren betrachten, sondern auch die Verhältnisse zwischen Uhren.

- Unterschiede zwischen Uhren können das Schaltverhalten beeinflussen

  z.B.: guard $x + 1 < y$



# Regionengraph: Beispiel



# Regionen und Zonen

- Regionengraphen sind endlich
- Eigenschaften, die als TCTL-Formeln ausgedrückt werden können, können auf Regionengraphen in endlicher Zeit geprüft werden

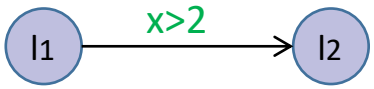
**Alle Eigenschaften, die in TCTL ausgedrückt werden können, bleiben in Regionengraphen erhalten.**

- Allerdings wächst die Anzahl der Regionen exponentiell mit der Anzahl an Uhren und der Größe der max. Konstanten
- Gezielte Reduktion der Anzahl an Äquivalenzklassen: **Zonen**

# Zonen: Intuition

**Idee**: Regionen mit gleichem Schaltverhalten sind äquivalent

**Beispiel**:




Äquivalente Regionen:

- [l1, **$x \leq 2$**]: (l1, $x < 1$), (l1, $x = 1$), (l1, $1 < x < 2$), (l1, $x = 2$)
- [l1, $x > 2$]: (l1, $x > 2$)
- [l2, $x > 2$]: (l2, $x > 2$)

Zonen Regionen

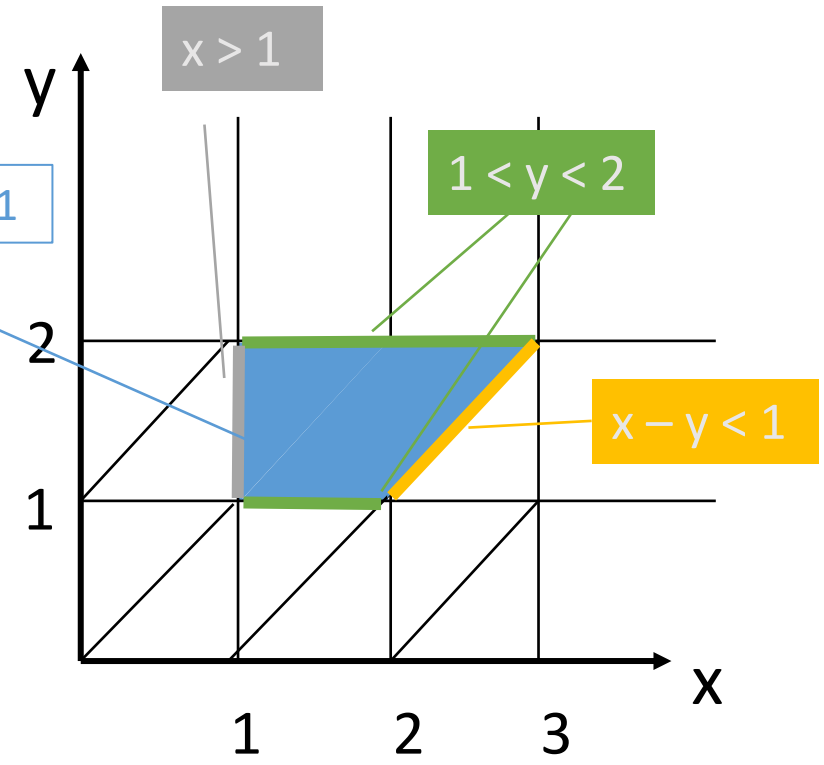
# Zonen

**Zone** = konvexe Konjunktion atomarer Uhrenbedingungen

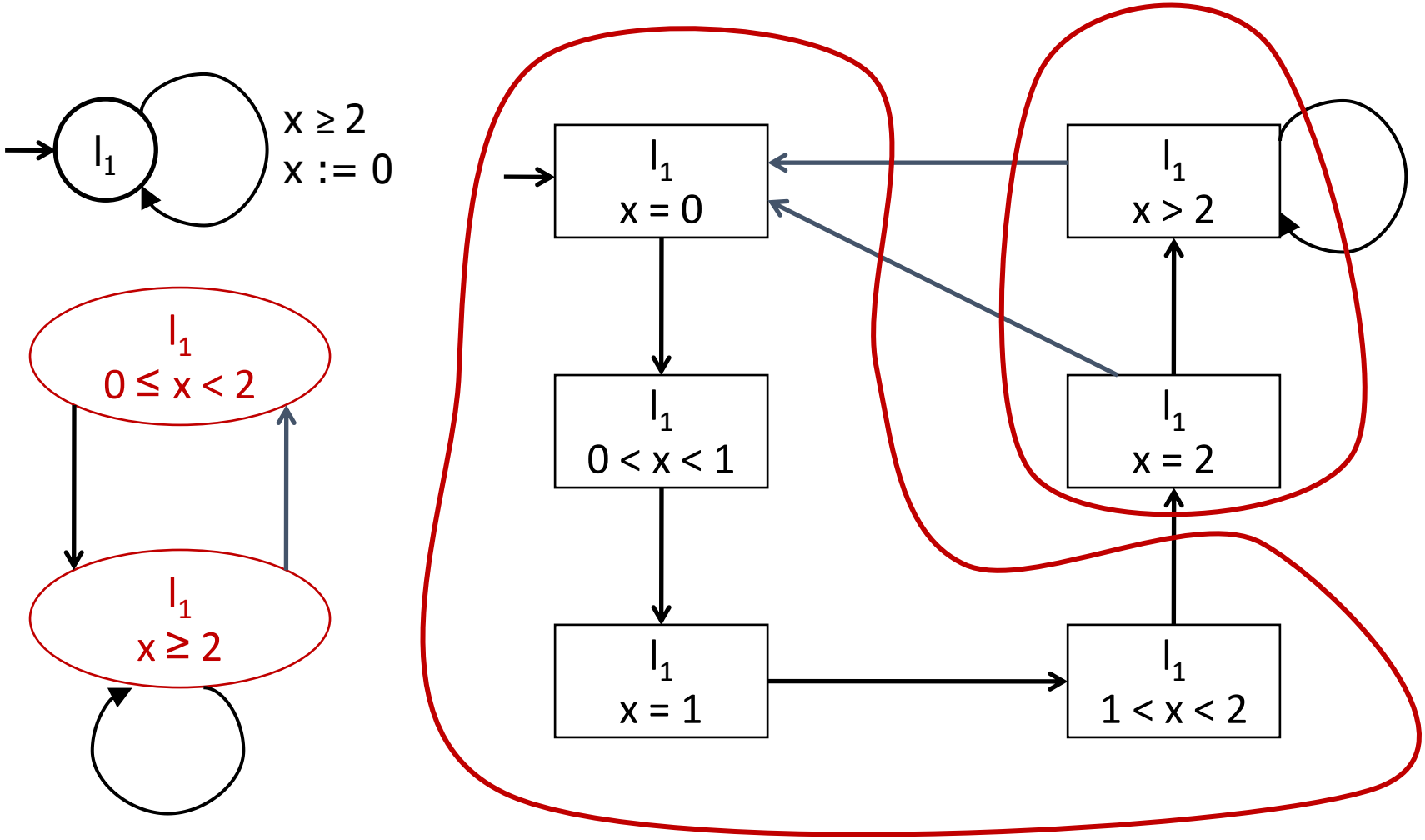
**Beispiel**:

$x > 1 \wedge 1 < y < 2 \wedge x - y < 1$

- Gezielte Vergrößerung der Regionen möglich, und damit Reduktion der Größe des Zustandsraums
- Regionen müssen nicht mehr explizit berechnet werden



# Zonengraph: Beispiel



# Fazit

- Verifikation von TAs mit dem UPPAAL Model Checker möglich
- Eigenschaften von TA können mit einer Temporallogik spezifiziert werden
- Das Transitionssystem eines TA bildet einen unendlichen Zustandsraum. Damit verifizierbar: Abstraktion von kontinuierlicher Zeit
  - Ziel: endlich viele Zustände
  - Methodik: Zonen als Äquivalenzklassen von Regionen, Regionen = Äquivalenzklassen von semantischen Zuständen (Location, Uhrenbelegung)

# Literatur


- R. Alur and T.A. Henzinger.
  **Real-time logics: complexity and expressiveness**.
  Information and Computation 104(1):35-77, 1993

- R. Alur and D.L. Dill.
  **A theory of timed automata**.
  Theoretical Computer Science 126:183-235, 1994